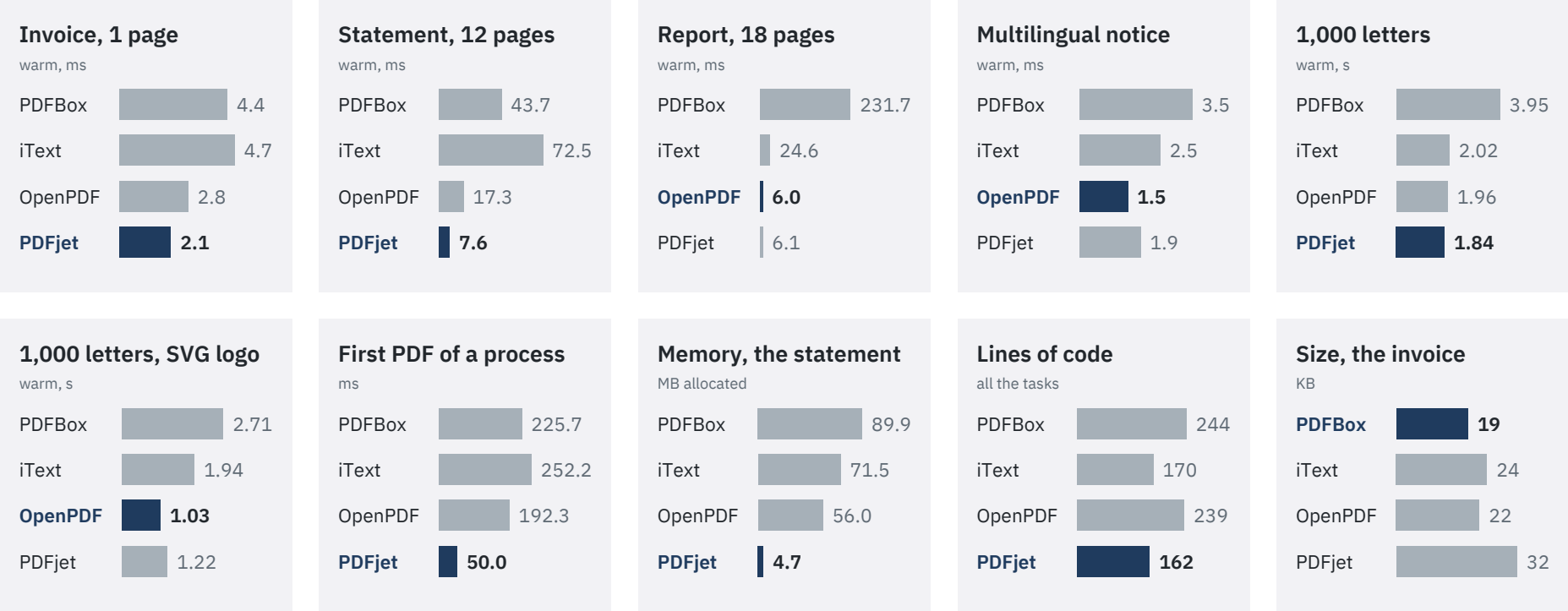
Generating PDFs in Java: four libraries compared

Five business documents and a batch of letters, made by each library the way its own documentation shows, every PDF checked, in IBM Plex Sans. Lower is better in every chart, and the best is in navy. Written by the author of PDFjet, with everything needed to run it again published; the tables, the method and where PDFjet loses are on the pages after this one.



When to use which

Apache PDFBox: to read, edit, render or extract text from existing PDFs, and for the smallest files. The most used of the four.

iText: for the most features in one library, and the shortest code for flowing documents, under the AGPL or a commercial license.

OpenPDF: for multilingual text, a large font and SVG with Batik, under the LGPL or MPL, on Java 21.

PDFjet: to generate business documents fast, with a fast first PDF, tagged PDF/UA, SVG and barcodes in one jar, under MIT, in Java, C#, Go and Swift.

I wrote PDFjet, one of the four libraries compared here, and have been writing it since 2001. For the last two months I have worked on it with Claude, an AI coding assistant. I commissioned this benchmark; Claude chose the documents, their content and the way each library is used, wrote the programs, ran them and wrote the tables. Both are reasons to be suspicious of the results, so everything needed to check them is published in github.com/edragoev1/pdf-benchmarks: the specification of the documents, every program, the scripts that run and check them, and the raw results.

If we have configured a library badly, or used it in a way its users would not, please tell us or open an issue, and we will fix the program and run the benchmark again.

When to use which

The four do different jobs well, and we recommend each of them where it is the better choice, as this test and their documentation show.

Apache PDFBox (Apache 2.0). If you chose it, this test does not say you chose wrong. Use it

- to read, change, render, print or extract text from PDFs that already exist: it is the strongest of the four at that, where PDFjet can only add to existing pages;
- when the size of the files matters most: its PDFs were the smallest of the four in every document;
- when you want the library the most published Java projects depend on.

Its gap, for generating documents: it has no layout and no built-in tagging, so text wrapping, tables and the structure tree of an accessible PDF are written by hand, which also made it the slowest here.

iText (AGPL-3.0 or commercial). Use it

- when you want the most features in one place: HTML to PDF, digital signatures, PDF 2.0 and PDF/UA-2, redaction and OCR, in its modules and add-ons, some of them commercial only;
- for flowing documents with the shortest code: its report and letters were the shortest programs of the four to write, as its Document flows text and photos onto pages by itself;
- when you want a company behind the library, with paid support, and the most questions answered on Stack Overflow.

OpenPDF (LGPL-2.1 or MPL-2.0). Use it

- when you want a classic document API under a weak copyleft license, on Java 21 or later;
- for multilingual text and for a large font: it was the fastest at the multilingual notice with both fonts, and at all three documents in Noto Sans; it also ties PDFjet at the report in Plex;
- for SVG with Apache Batik: it drew the letters with an SVG logo the fastest of the four.

PDFjet (MIT). Use it

- when you generate business documents: it was the fastest or tied for the fastest at the invoice, the statement, the report and the letters with IBM Plex Sans, and allocated the least memory for most of them;
- when the first PDF of a process counts, as in a command-line tool or a serverless function: its first PDF took about a quarter of the time of the next library's;
- when you want the same library in C#, Go and Swift as well as Java;
- when you want tagged PDF/UA-1, SVG, barcodes, Markdown and PDF/A in one jar, with no other dependency.

The libraries

Library	Version	License	Java
PDFjet (core)	9.1.5 (main at 84e73b5f until the release)	MIT	8 and later
Apache PDFBox	3.0.8	Apache 2.0	8 and later
iText Core	9.8.0	AGPL-3.0 or commercial	8 and later
OpenPDF	3.0.5	LGPL-2.1 or MPL-2.0	21 and later

The newest releases on Maven Central on 26 October 2026. For the letters with an SVG logo, the modules the others need to draw SVG, each at its newest release: iText's svg module 9.8.0, Apache Batik 1.20, and for PDFBox the pdfbox-graphics2d bridge 3.0.5.

Each library is used the way its own documentation shows: iText's and OpenPDF's `Document` for flowing text, PDFBox's content streams with lines wrapped by hand, as it has no layout of its own, and PDFjet's TextFrame and Table, and for the report TextFlow, new in 9.1.5, which flows headings, paragraphs and photos onto pages as a Document does. Fonts are kept the way a server would keep them: iText and OpenPDF with their own default caches, PDFBox with its TrueType font parsed once and reused, as its API allows; PDFjet has no font cache and reads its fonts for each document.

What was measured

Six tasks, specified exactly in SPEC.md, the same text and the same layout in every library:

- **invoice**: 1 page, the PDFjet logo, two address blocks, a table of 15 rows, totals.
- **statement**: 12 pages, a table of 600 transactions with running footers "Page n of N".
- **report**: 18 pages, 120 paragraphs of flowing text and 4 photos, JPEGs of 1870 × 2420 pixels, 1.8 MB in all.
- **multilingual**: 1 page, a notice in eight languages: English, French, German, Polish, Turkish, Greek, Bulgarian and Ukrainian.

- **letters**: 1,000 one-page letters with the logo, a PNG, timed as the batch.
- **letters with an SVG logo**: the same letters with the same logo as SVG.

All in IBM Plex Sans, embedded as subsets; the invoice, the report and the notice again in Noto Sans, a much larger font.

Method. Each library in its own fresh JVM, in a random order, three rounds. Each task is warmed up for at least 20 runs and 3 seconds, then timed over 15 runs (the letters: 2 warm-up batches, 5 timed); the figure is the median of the three round medians. An asterisk (*) marks a figure whose rounds differ by more than 5%. The first PDF of a fresh process is the median of 5 processes. OpenJDK 25, AMD Ryzen 5 5600G, Linux, nothing else running. After the run every PDF is checked, and the log is published: MuPDF opens it with no warning, it has the expected pages, the check strings of SPEC.md are in its text (the last sentence of every language for the notice), both fonts are embedded as subsets, and qpdf --check passes.

Results

Time per document, warm, ms (the letters: the batch of 1,000, s). In every table lower is better, faster, less memory, smaller files, fewer lines, and the best of the four Java libraries is in bold. The last three columns are PDFjet in C#, Go and Swift, shown with them but not ranked against them.

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice, 1 page	4.4 *	4.7 *	2.8 *	**2.1**	2.0	1.5 *	2.1
statement, 12 pages	43.7	72.5 *	17.3	**7.6**	8.1	4.8	11.3
report, 18 pages	231.7	24.6 *	**6.0**	6.1 *	9.0	6.3	12.9
multilingual, 1 page	3.5	2.5	**1.5**	1.9	1.7	1.1 *	1.7
1,000 letters, s	3.95	2.02	1.96	**1.84**	1.74	1.50	1.63
1,000 letters, SVG logo, s	2.71	1.94	**1.03**	1.22	1.17	1.04	1.69

To draw the SVG logo the others need another module or library, and each was given the one its users would choose:

Library	to draw SVG it needs
PDFBox	Apache Batik and pdfbox-graphics2d
iText	iText's svg module
OpenPDF	Apache Batik
PDFjet	nothing else

With Noto Sans, a large font, ms:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	18.9 *	5.9 *	**3.6 ***	5.0	4.7	2.7 *	4.4
report	1,350	28.0	**6.8 ***	8.8	12.0 *	7.6	15.1
multilingual	22.1	4.2	**2.0**	4.5	4.4	2.4 *	2.9

The first PDF of a fresh process, the invoice, as a command-line tool, a serverless function or a server just started pays it. The peak memory is that of the whole process, mostly the JVM itself:

	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
the invoice, ms	225.7	252.2	192.3	**50.0**	49.1	4.3	4.5
the whole process, s	0.28	0.31	0.24	**0.08**	0.10	0.00	0.01
its peak resident memory, MB	87	91	84	**54**	42	10	28

Memory allocated per document, MB:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	4.0	3.6	2.2	**1.6**	1.5	2.7	—
statement	89.9	71.5	56.0	**4.7**	5.5	4.8	—
report	232.6	26.8	13.4	**12.1**	12.6	9.1	—
multilingual	5.0	1.8	**0.6**	1.1	1.1	1.3	—
a letter	5.0	1.4	**1.0**	1.5	1.4	2.2	—

The size of the PDFs, KB:

Document	PDFBox	iText	OpenPDF	PDFjet Java	PDFjet C#	PDFjet Go	PDFjet Swift
invoice	**19**	24	22	32	32	32	34
statement	**42**	62	59	70	70	70	79
report	**1,816**	1,824	1,817	1,829	1,829	1,830	1,833
multilingual	**22**	30	25	35	35	36	38
a letter	**13**	18	17	26	26	27	28

Lines of code of each program, without blank lines and comments; "helpers" are the methods a library's tasks share (fonts, cells, footers, word wrapping), counted in the total. The letters with an SVG logo are not in the total:

	invoice	statement	report	letters	multilingual	helpers	total
PDFBox	**28**	26	53	23	24	90	244
iText	39	30	**22**	**17**	**17**	45	170
OpenPDF	57	37	32	25	23	65	239
PDFjet Java	31	**25**	32	20	**17**	**37**	**162**
PDFjet C#	32	25	32	20	17	35	161
PDFjet Go	36	26	35	23	20	35	175
PDFjet Swift	29	22	31	18	15	45	160

Where PDFjet loses

- **Multilingual text:** OpenPDF was faster at the notice in eight languages with both fonts, and with Noto Sans iText was too. PDFjet's font subsets keep every glyph slot of the font, which grows with the scripts a document uses.
- **A large font:** OpenPDF was faster at all three documents in Noto Sans: the invoice, the report and the notice.
- **The report in Plex:** a tie with OpenPDF, its rounds within PDFjet's own spread.
- **SVG:** OpenPDF with Batik drew the letters with an SVG logo faster than PDFjet draws them by itself.
- **The size of the PDFs:** PDFjet's were the largest of the four. Its font subsets keep every glyph slot of the font and the font's cmap and name tables, which PDFBox renumbers and drops; PDFjet also writes XMP metadata in every PDF, and PDFBox compresses its objects in object streams.
- **Code for flowing documents:** iText's report and letters were shorter to write than PDFjet's (22 and 17 lines against 32 and 20). Over all the tasks PDFjet's programs were the shortest, 162 lines to iText's 170.

What the others do that PDFjet does not

From each library's documentation, not tested here.

Feature	iText	OpenPDF	PDFBox	PDFjet
Extract text	yes	basic	yes	no
Render a page	add-on [1]	module [2]	yes	no
Edit a page	yes	yes	yes	adds only
Signatures	module	yes [3]	yes	no
Certificate encryption	yes	yes [3]	yes	passwords

Feature	iText	OpenPDF	PDFBox	PDFjet
HTML to PDF	module	module	no	Markdown
Hyphenation	module	unverified	no	soft hyphens
PDF 2.0	yes	yes [4]	reads it	no
PDF/UA-2	yes	no	no	no
Redaction, OCR, XFA	add-ons [1]	no	no	no

[1] commercial for some (pdfRender, pdfXFA). [2] openpdf-renderer. [3] with Bouncy Castle. [4] per its README. PDFjet makes PDF 1.7 and PDF/UA-1.

And beyond features: far more people have worked on PDFBox, iText and OpenPDF than on PDFjet, and far more people use them, with more answers on the web for every question; iText also has a company behind it with paid support.

PDFjet in C#, Go and Swift

PDFjet is also written in C#, Go and Swift: the same classes and the same public API in each, and the same documents laid out the same way, which an automated check compares on every change. The last three columns of the tables above are the same documents made by each of them, for anyone choosing which one to use, and, as an aside, because the same work done by the same library in four runtimes says something about the runtimes too.

Compression is the biggest difference between them. Every PDF compresses its pages, and each port compresses them differently:

Port	Deflate	Level
Java	zlib (java.util.zip), native C	5
C#	zlib (ZLibStream), native C	6, .NET's Optimal; it has no 5
Go	compress/zlib, written in Go	6
Swift	PDFjet's own, written in Swift	its own, searching less than zlib

Where the time of the 12-page statement goes, from a profile of each port at its steady state (perf, a long run so that the JIT runtimes have finished compiling):

Share of the time	Java	C#	Go	Swift
compressing	59%	43%	32%	16%
the runtime: GC, reference counting, allocating, copying	13%	18%	26%	30%

Share of the time	Java	C#	Go	Swift
strings, the language's own	—	—	—	14%
PDFjet's own code, and the rest	28%	39%	42%	40%

(Measured 26 October with every port at level 6, before Java went to level 5; to be measured again with 9.1.5.)

So in Java more than half of the time making a statement is zlib, and how fast Java is here says as much about zlib as about the JVM. Go's own Deflate is fast enough that compressing is a third of its time. Swift spends the least on compressing, with an encoder that searches less and writes larger files, and the most on its runtime: reference counting, exclusivity checks and its String type take more of its time than compressing does. C# spends about a tenth of its report copying and zeroing memory buffers for the photos, where Java spends a twentieth, which is part of the gap between them in that document.

The ports are not Java in other syntax. Each is written for its language, with its own standard library and idioms, and the results show it: one design translated would rank the same in every document, and these do not. Each port is its own code, so a difference can come from a port as well as from its language, as compression shows above. If you find an omission that puts your favorite language at a disadvantage, a slow way of doing something where the language has a faster one, please tell us: we will be happy to adopt it in the library and run the test again.

Reproduce it

Everything is in github.com/edragoev1/pdf-benchmarks: SPEC.md, the text of the notice, the logo and the photos, the program of each library and port, the run and check scripts, and the raw results with their logs. java/run.sh runs the four Java libraries (about 30 minutes) and java/check.sh checks every PDF; dotnet/, go/ and swift/ run PDFjet's other ports the same way; make-tables.py makes every table of this report from the results.

Judgment calls

- The table rows of every library are 13.2 pt high, PDFjet's default for IBM Plex Sans at 9 pt; iText's and OpenPDF's cell padding was adjusted to match it, as without it their page counts differ.
- OpenPDF uses `setStrictImageSequence(true)`, or it moves text above a photo that goes to the next page.
- iText and OpenPDF use their large-table modes for the statement, as their documentation shows for long tables. iText's ordinary Table was timed too, and was slower (about 106 ms against 72).
- PDFBox draws one rule under each table row and one header rectangle, where the others draw a border under each cell: slightly less work for PDFBox.
- PDFBox applies the font's ligatures (GSUB) to its text, which the others do not; with Noto Sans, whose tables are large, this is the likely cause of its slow report, which allocates 2.8 GB, though it was not profiled.

- PDFjet's Noto report is 21 pages to the others' 20: it spaces lines by the font's own height.
- PDFjet and iText write text as hex strings (`<48656C6C6F> Tj`), OpenPDF and PDFBox as literal strings (`(Hello) Tj`). Hex takes twice the bytes before compression, but after it the content of a page is about the same size in all four, 1.3 to 1.5 KB, so it is not where PDFjet's larger files come from. Both are correct, and current viewers read both; hex needs no escaping of parentheses, backslashes or line ends, which is why PDFjet uses it.